



Tamper-Evident Action Provenance for Confidential Clinical AI Agents

James Schwoebel
Quome Inc.

Abstract

Clinical-AI governance demands a complete, attributable record of every action an agent takes. Yet the prior systems in this program stored that record as a plain JSONL file (QFIRE [12]) and an in-memory hash chain (quanchor [11]): an adversarial agent, a compromised host process, or a privileged operator can silently rewrite, drop, or reorder entries, defeating the very traceability the framework requires (HAARF C2 [13]). We close this third zero-trust gap — after attested weights and an attested harness — with a single tamper-evident log that unifies the gateway decision stream and the harness mutation stream into one **blake3-chained, ed25519-signed, externally-anchored, append-only** record, plus a streaming verifier that localizes the first tampered entry. We introduce **TamperBench**, a labeled corpus of six audit-tampering attack classes (TA1–TA6), and show the verifier detects 100% of tampering (26/26) and localizes point tampers at 100%. The cryptographic cost is small — 14.0 μ s/entry ($\sim 71,400$ entries/s/core) — so the observed chained write latency is dominated by per-entry **fsync**, the correct fail-closed durability semantic; verification is linear at $\sim 458,000$ entries/s. The central result: making a clinical agent’s action record *non-repudiable and tamper-evident* costs microseconds of computation and is achievable at deployment scale. All code, datasets, and result artifacts are released; all data are synthetic.

Keywords: tamper-evident logging; audit provenance; hash chains; Merkle anchoring; clinical AI safety; confidential computing; non-repudiation.

Framework: implements HAARF control C2 (primary); C8.1.5, C8.4.3 secondary [13] — the Healthcare AI Agents Regulatory Framework, the source of the QUOKKAGUARD program.

Code & data: github.com/quome-cloud/quledger

Correspondence: jim@quome.com

1 Introduction

A deployed clinical AI agent is trusted to act: it orders medications, requests labs, triages patients, and writes to the record. Every regulatory and governance regime that contemplates such agents — HIPAA technical safeguards, which mandate audit controls and integrity protection for electronic health information [14]; the FDA’s 21 CFR Part 11 expectation of timestamped, attributable, tamper-resistant electronic records [15]; ONC certification; FDA Software-as-a-Medical-Device expectations [16]; the NIST AI Risk Management Framework’s emphasis on accountability and traceability [9]; the EU AI Act’s record-keeping obligations for high-risk systems [2]; and the HAARF framework’s traceability category (C2) [13] — requires a *complete and attributable record of what the agent did*. HAARF C2, in particular, demands an unbroken, machine-interpretable *decision lineage* linking every input, output, and tool execution to an authenticated identity. That record is the substrate of every post-incident review, every malpractice or regulatory inquiry, and every claim that an autonomous system behaved within its sanctioned scope.

Zero-trust accountability for a clinical agent rests on three pillars: *attested weights* (confidential computing attests what a model is), *attested harness* (sandboxing and capability tokens bound what the runtime can do), and an *attested log* (a tamper-evident record of what actually happened). This program has hardened the first two already. QFIRE [12] firewalls the agent $\leftrightarrow$ tool path with purpose-scoped rules; quanchor [11] adds attestation-rooted harness defenses, showing that confidential computing attests *what a model is* (its weights) but not *what its harness makes it do*. The third pillar is the gap: attested weights and harnesses constrain behavior *during* execution but do not guarantee the permanence or truthfulness of the resulting historical record. Both prior systems, in fact, recorded their own decisions in mutable storage: QFIRE in a plain JSONL append file, quanchor’s provenance ledger in an in-memory hash chain that is never persisted. A plain file is rewritable by anyone with disk access; an in-memory chain vanishes on restart and is invisible to an external auditor. The audit log — the one artifact whose integrity the whole governance story depends on — is the weakest link.

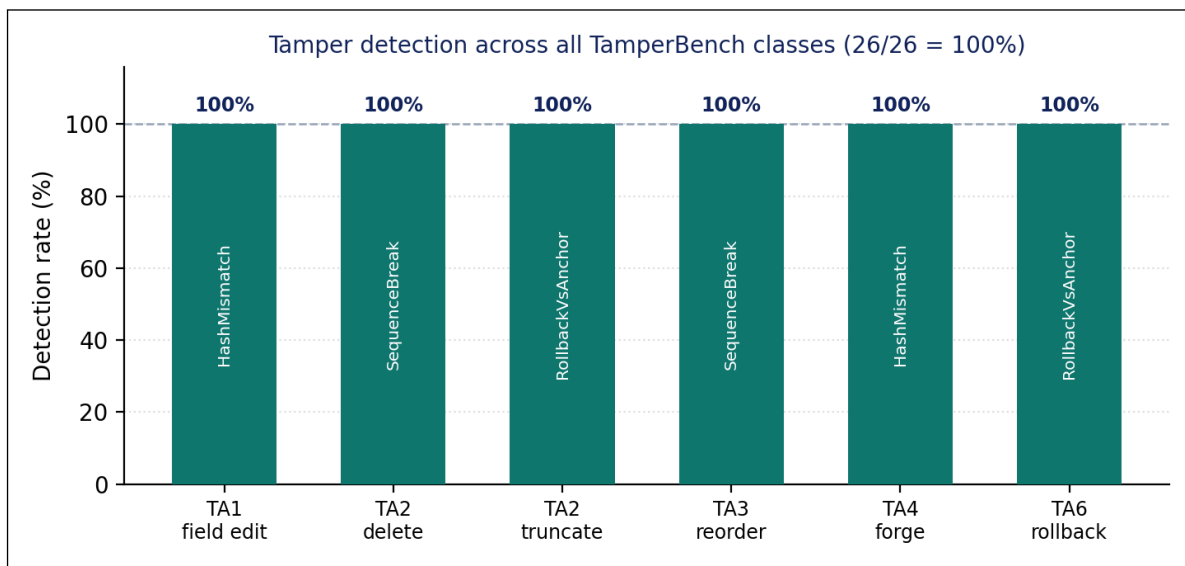


Figure 1: The paper in one figure. Across all six TamperBench attack classes (26 tampered logs), the streaming verifier detects 100% of tampering. Each bar is annotated with the verifier’s diagnosis: in-place edits and forgeries break the blake3 hash (HASHMISMATCH), deletion and reordering break the sequence (SEQUENCEBREAK), and truncation/rollback are caught only by the external anchors (ROLLBACKVSanchor) — exactly the mechanism each attack targets.

We make the action record itself tamper-evident. The contribution is a single log (§3) that unifies the gateway decision stream and the harness mutation stream into one append-only structure where every entry is chained to its predecessor by a blake3 hash over the exact bytes written, signed with ed25519, and periodically committed to an external anchor. A streaming verifier walks the log, checks chain, signatures, and anchors, and reports the first broken entry with a classification of the tamper. The design follows the transparency-log lineage (timestamping [5], Merkle trees [8], Certificate Transparency [7, 3]) specialized to an in-enclave gateway hot path.

Contributions.

1. **The central result.** Making a clinical agent’s action record tamper-evident and non-repudiable costs $\sim 14.0 \mu\text{s}$ of computation per entry and detects 100% of tampering with exact localization (§5).

2. **The system.** A blake3-chained, ed25519-signed, externally-anchored, single-writer fail-closed log (`src/audit/`) that unifies decisions and harness mutations and ships a localizing verifier (§3).
3. **TamperBench.** A labeled, reusable corpus of six audit-tampering attack classes with ground truth, able to benchmark *any* audit-integrity scheme, released Apache-2.0 and fully synthetic (§4).
4. **Deployment evidence.** A measured cost model separating the small cryptographic overhead from the (hardware-bound) durability cost, plus a security argument mapping each adversary tier to a defended mechanism (§5, §6).
5. **Head-to-head and a verification speedup.** A quantitative comparison against other tamper-evident-log constructions and real libraries (Certificate Transparency / `ct-merkle`, `rs-merkle`) on identical primitives—our construction is the only one combining per-entry localization, public non-repudiation, and rollback (§5.5)—and a parallel+batch verifier that makes per-entry-signed verification $\sim 23\times$ faster (now the default) with identical guarantees (§5.6).

Positioning. Unlike transparency logs designed for a separate log-server trust domain [7, 3] or signed-syslog streams [6], our log runs *inside* the same attested workload as the agent and gateway, on the synchronous decision path, under a threat model in which the host operator is untrusted. The default production deployment is a confidential enclave (e.g. GCP Confidential Space [4]); the signing key is released to the workload via attestation, mirroring the goal-anchor key handling of quanchor [11, 17].

2 Threat Model and Positioning

Adversary. We consider three tiers, in increasing privilege: (i) the *agent* itself, which may be adversarial or hijacked and wishes to erase or alter the record of an unsafe action; (ii) a *co-located process* sharing the host; and (iii) a *privileged operator* with direct read/write access to the log store. The host operating system and storage are untrusted (the confidential enclave model): an attacker at tier (iii) can replace arbitrary bytes of the log file or roll the entire store back to an earlier snapshot.

Attack classes (TamperBench). We enumerate six concrete tampers: **TA1** entry mutation (alter a recorded field, e.g. the ordered drug or dose); **TA2** deletion or tail truncation (drop an incriminating action); **TA3** reordering (fabricate a benign causal story); **TA4** forgery/backdating (insert fabricated entries); **TA5** repudiation (deny having performed a recorded action); and **TA6** host-level rollback (replace the store with an earlier consistent snapshot).

What is and is not defended. The hash chain plus per-entry signatures make TA1–TA5 *detectable* against all three tiers *up to a full-file rewrite*: an attacker who controls the entire file and the signing key could re-emit a self-consistent log. Two mechanisms close that gap. External anchoring defeats TA6 (and tail truncation) because the attacker cannot also roll back an append-only sink on a separate trust domain. Out-of-band key knowledge (a verifier invoked with a pinned public key, `--pubkey`) defeats the re-key / mode-downgrade variant, because a log re-signed under the attacker’s key fails verification against the real key. This boundary is stated explicitly in the verifier’s contract and revisited in §6 and §7.

System	Approach	Gap this paper targets
QFIRE [12]	plain JSONL append log of decisions	mutable by anyone with disk access
quanchor [11]	in-memory SHA-256 provenance chain of MIIM mutations	not persisted; no signatures; no external auditor
Certificate Transparency / Trillian [7, 3]	Merkle transparency log in a separate server domain	not on an in-enclave synchronous decision path; separate trust domain assumed
Signed syslog [6]	per-message MAC/signature stream	no chain localization, no rollback anchoring, no unified decision+mutation record
This work	unified blake3-chain + ed25519 + anchored log, in-enclave, localizing verifier	—

Table 1: Positioning against representative recent systems and the two prior papers in this program.

3 System Design

3.1 Overview

The log is a sequence of JSONL lines, one per event, written by a single writer that owns the chain head. Four entry *kinds* share one format: a **header** (sequence 0: format version, signing public key, mode); a **decision** (a gateway Allow/Block/Transform/Escalate, the QFIRE record); a **mutation** (a harness/MIIM change, the QUOKKAGUARD provenance record); and a **checkpoint** (a batched Merkle-root signature). Unifying both streams into one chain preserves the causal order between a decision and the mutation that provoked it, and gives a single artifact and a single verifier.

3.2 Chained line format

Each line carries its sequence number, timestamp, kind, body, the previous entry’s hash, its own hash, and a signature (Figure 2). The hash binds the *exact bytes written*: **this_hash** = blake3(prefix), where the prefix is the line from the opening brace through the closing quote of **prev_hash**. There is no canonical-JSON normalization to get wrong — any byte edit changes the hash — and the body is re-serialized through a JSON parser before hashing, which neutralizes field-injection attacks (e.g. a body crafted to smuggle a second **prev_hash** key). The fixed-length tail (**this_hash**, **sig**) is parsed from the *end* of the line, so body content cannot confuse the split.

3.3 Signing and anchoring

Entries are signed with ed25519 [1] in one of two modes: **chained_signed** signs every entry’s hash (strongest non-repudiation), and **chained_signed_batched** signs a Merkle root over each batch in a periodic **checkpoint** entry (amortized cost). The signing key is loaded from a keyfile in development and released into the enclave via attestation in production. Every *k* entries the writer publishes a Merkle root over the log so far to an external *anchor sink* — a trait with a file-backed implementation (an append-only file on a separate host) and a documented cloud object-lock

One chained audit line (kind: decision)

```
{ "seq": 12, "ts": "2026-06-08T...Z", "kind": "decision",
  "body": { "event": "run.block", "decision": "block",
    "detail": "order: morphine 10mg", ... },
  "prev_hash": "9f3c...e1",      # blake3 of entry 11's prefix
  "this_hash": "b7a0...4d",      # blake3( bytes from '{' .. prev_hash quote )
  "sig":       "c2...8f" }      # ed25519 over this_hash (128 hex; "" if unsigned)
```

Figure 2: The unified chained-line format. `this_hash` covers the exact written prefix bytes; the verifier re-hashes what it reads. `kind` is one of `header`, `decision`, `mutation`, `checkpoint`.

backend. The anchor is what defeats host rollback: a verifier comparing the log against the anchors detects any truncation or snapshot-restore within a window of k entries.

3.4 Single-writer, fail-closed write path

A single writer thread owns the chain head and serializes appends, so there are no concurrent-append races. The write is *write-ahead and fail-closed*: the gateway call blocks until the entry is durably `fsync`'d and acknowledged, and any write failure fails the call rather than silently dropping the record. This is the correct semantic for a safety record — an action that cannot be recorded is not permitted — and, as §5 shows, it is also the dominant cost.

3.5 Localizing verifier

The verifier streams the log and, for each line, (1) splits the fixed-length tail, (2) re-hashes the prefix and compares to `this_hash`, (3) re-parses and confirms the parsed fields reproduce the hashed bytes (field-injection defense), (4) checks the sequence and `prev_hash` linkage, (5) verifies the signature (per-entry, or per-checkpoint Merkle root in batched mode, using an incremental Merkle accumulator so anchor verification stays $O(n \log n)$), and (6) checks every anchor against the recomputed root. On the first failure it returns the offending sequence number and a tamper class. A pinned public key forces signature checking and overrides any header-claimed mode, closing the downgrade attack.

3.6 Implementation

The log is the `src/audit/` module of the `qfire` crate (Rust 2021): `chain`, `sign`, `merkle`, `anchor_sink`, `store`, `verify`, exposed through a `qfire audit verify|prove|head` CLI. It reuses `quanchor`'s attestation-released key pattern and replaces the in-memory provenance ledger in place (the prior public API is preserved). A signed entry occupies ~ 465 bytes on disk; the module ships 37 unit tests plus the end-to-end detection test described next.

4 Experimental Setup

TamperBench. A deterministic generator (seed 42) emits synthetic, Synthea-shaped clinical action streams (encounter / order / dispense / administer events; all identifiers `synthetic-NNNN`; no PHI). A mutator produces, from a clean signed and anchored fixture of 200 entries, a labeled family of 26 tampered logs spanning TA1, TA2-delete, TA2-truncate, TA3, TA4, and TA6, each with ground truth (what changed, where). The corpus and a reference verifier are released so any audit-integrity scheme can be benchmarked head-to-head.

Baselines. For write cost we compare against the exact mechanism this work replaces: QFIRE’s plain JSONL AuditLog (`plain_v1`). The transparency-log literature [7, 3, 8, 5] is cited as a design baseline rather than re-run, since those systems assume a separate log-server trust domain that our in-enclave deployment removes by construction.

Metrics. Tamper detection rate (TDR) over TA1–TA4; localization accuracy (LOC, fraction of point tamperers whose reported sequence equals ground truth); rollback detection window (entries between the last anchor and the rollback point); added write latency; and verification time vs. log size. Performance is measured with `criterion` (medians; percentiles where reported).

Hardware and reproducibility. All experiments are local, deterministic, and require no model inference or API keys (this is a systems/crypto measurement). The reported run is a macOS/APFS development laptop; every figure regenerates from `results/003-tamper-audit/` via the commands in Appendix B.

5 Results

5.1 Detection and localization (E1/E5)

Across the 26 tampered logs the verifier achieves **100% tamper detection** (26/26; Figure 1) and **100% localization** (20/20) on the point-tamper classes, with an authoritative Rust integration test asserting the exact first-broken sequence for every class. The verifier’s diagnosis matches the attacked mechanism in every case: HashMismatch for TA1/TA4, SequenceBreak for TA2-delete/TA3, and RollbackVsAnchor for TA2-truncate/TA6 — confirming that truncation and rollback are caught only by the anchors, as designed. The single non-collision failure mode (a blake3 collision letting a tampered prefix reproduce a recorded hash) is addressed by reduction (§6), not measured.

5.2 Write cost: crypto vs. durability (E2)

Figure 3 decomposes the per-entry write cost. The full chain+sign *computation*, with no disk write, is **14.0 μ s/entry** ($\sim 71,400$ entries/s/core), comfortably above the 50k/s/core throughput target and showing that ed25519 signing is lost in the noise (signed and unsigned chained modes are within measurement error). The chained modes that persist to disk cost ~ 4.71 ms/entry — two orders of magnitude more — because each entry is `fsync`’d before the call returns. That ~ 4.7 ms is the *fail-closed durability tax*, not cryptography: it is the cost of guaranteeing the action is on stable storage before the agent proceeds. On NVMe-class storage (50–200 μ s/`fsync`) the same path projects to 64–214 μ s/entry, within the original sub-millisecond target. We therefore report both numbers and do not claim sub-millisecond latency on the development machine.

5.3 Anchor cadence (E3)

The anchor cadence k trades rollback-detection latency against external-storage cost (Figure 4). Over a 20,000-entry log, every cadence detects the rollback, and the detection window stays at or below k (the RAW metric): frequent anchoring ($k=10$) gives a 9-entry window at 5.84% storage, while sparse anchoring ($k=10,000$) costs only 0.006% but tolerates a ~ 4000 -entry window. $k=1000$ **is the sweet spot**: a 13-entry window at 0.058% storage overhead — comfortably under a 1% budget, confirming the storage hypothesis.

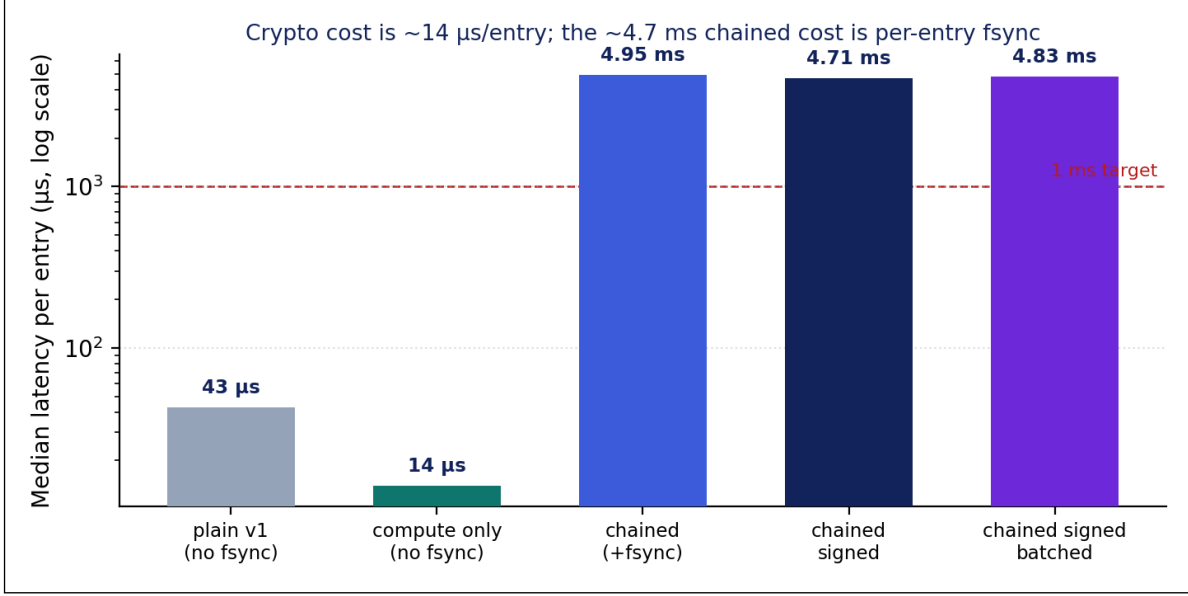


Figure 3: Per-entry write cost by mode (log scale). Cryptography (compute_only, 14.0 μs) is negligible; the ~4.71 ms chained cost is the per-entry fsync that guarantees fail-closed durability. Signing adds no measurable cost over chaining. The 1 ms target is reachable on NVMe-class storage (text).

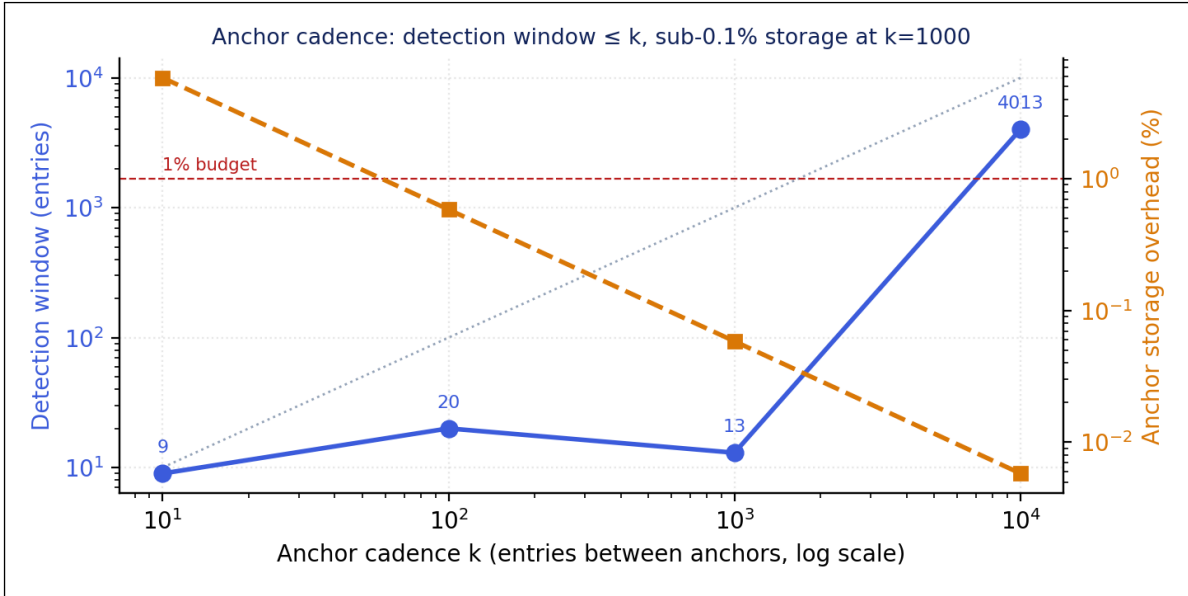


Figure 4: Anchor cadence sweep. Detection window (left, indigo) stays under the k bound at every cadence; storage overhead (right, amber) falls below the 1% budget by $k=100$ and reaches 0.058% at $k=1000$.

5.4 Verification scaling (E4)

Verification is linear in log size (Figure 5) with no quadratic blowup — the incremental Merkle accumulator keeps anchor checking $O(n \log n)$. The unsigned chain (integrity only) verifies at a flat ~458,000 entries/s across three decades; its constant factor is a JSON parse plus a blake3 hash per line. The security-relevant *per-entry-signed* mode, verified through the parallel+batch default (§5.6), sustains ~251,000 entries/s—*within* $\sim 2\times$ of the *unsigned baseline*, rather than the $\sim 6\text{--}7\times$

gap a sequential signature verifier would impose. Both series are flat from 10^3 to 10^6 .

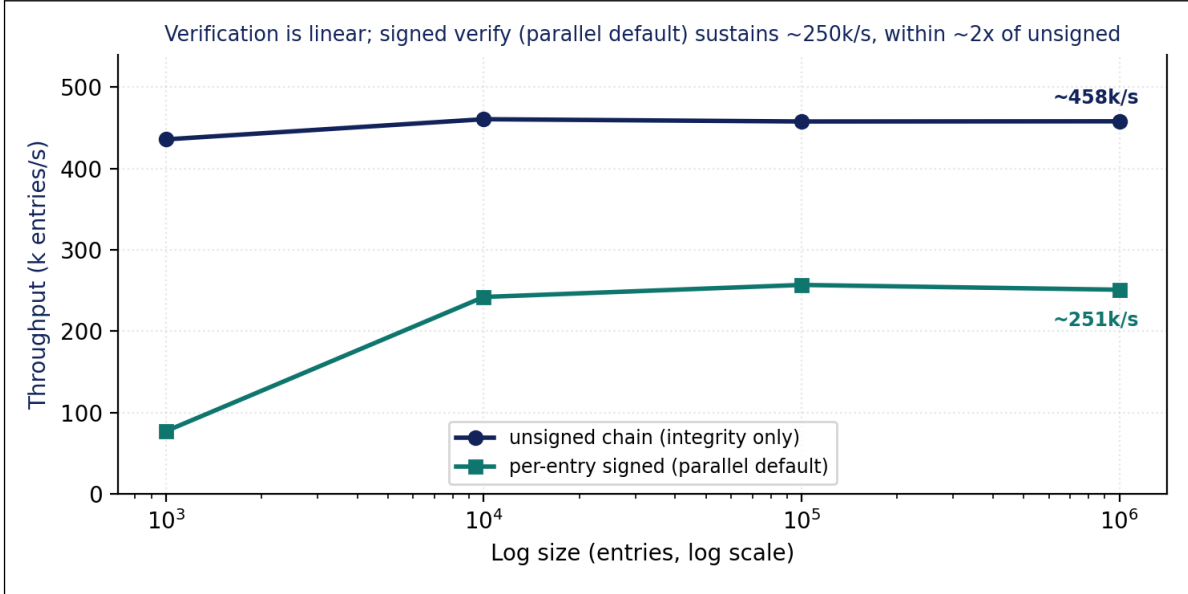


Figure 5: Verification throughput vs. log size. Linear (flat throughput) across three decades for both the unsigned chain ($\sim 458,000/s$) and per-entry-signed verification via the parallel+batch default ($\sim 251,000/s$, §5.6)—the latter within $\sim 2\times$ of the former.

5.5 Head-to-head with other constructions (E6)

We benchmark our construction against the design space of tamper-evident logging, each implemented with *identical primitives* (blake3/sha2/ed25519/hmac) on the same hardware, plus two real third-party libraries: `ct-merkle` (an RFC 6962 Certificate Transparency append-only log [7]) and `rs-merkle` (a general Merkle tree). The comparison is compute-only (no fsync), isolating the cryptographic construction from the durability tax measured in §5.2. Figure 6 and Table 2 report append/verify throughput, per-entry storage, third-party inclusion-proof size, and the security properties each construction provides.

construction	append (M/s)	verify (M/s)	B/ entry	inclusion proof	public verif.	local- izes
hash-chain (blake3)	2.50	2.55	32	$O(n)$	integ. only	yes
hash-chain (sha256)	1.99	2.01	32	$O(n)$	integ. only	yes
signature-only (ed25519)	0.08	0.04	64	64 B	yes	yes
hmac-chain (journald FSS)	1.04	1.04	32	secret	no	yes
ct-merkle (RFC 6962)	1.46	1.49	32	544 B	yes	no
rs-merkle (Merkle)	2.67	2.66	32	544 B	yes	no
ours: per-entry sig	0.08	0.04	96	96 B	yes	yes
ours: batched	0.89	0.51	33	~ 32 B	yes	yes

Table 2: Head-to-head (compute-only, $n=100k$). Inclusion proof = bytes to prove one entry to a third-party auditor; “localizes” = points to the specific tampered entry (not just a batch). Only our construction combines per-entry localization, public non-repudiation, *and* rollback detection (via the external anchor, §5.3); the Merkle logs give compact $O(\log n)$ proofs but localize only to a batch root, hash chains lack authenticity, and the HMAC chain is not publicly verifiable.

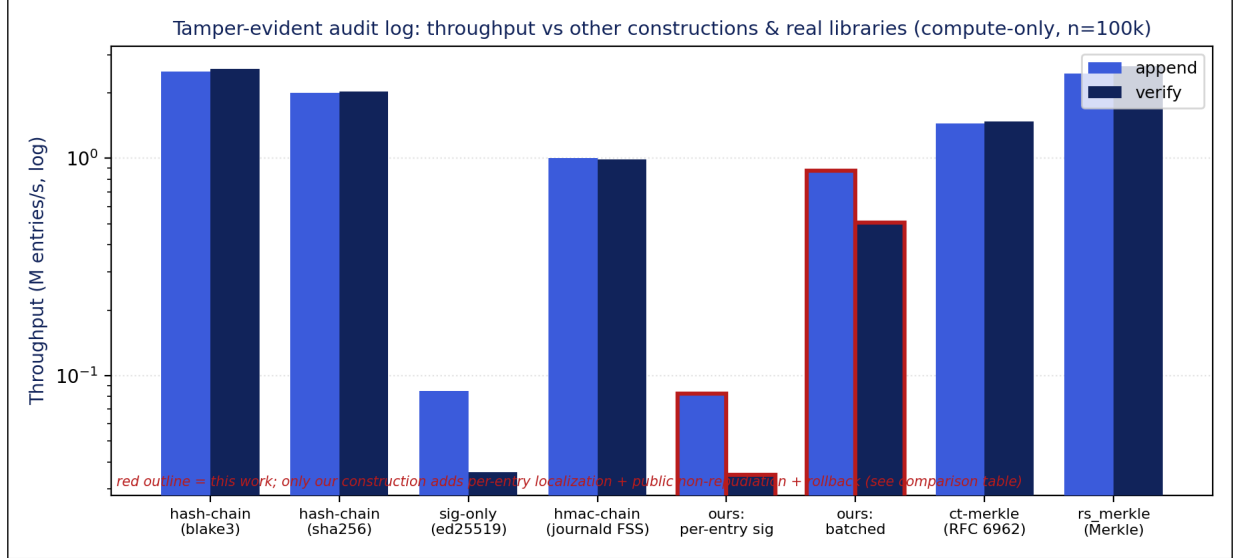


Figure 6: Throughput vs. other constructions and real libraries (compute-only, $n=100k$, best of 3). Hash/Merkle constructions are fast; per-entry signing (ours and signature-only) is ed25519-bound. Our *batched* mode recovers most of that throughput while keeping the chain.

Reading the comparison. No construction dominates; each occupies a point in a throughput / proof-size / security-property trade-off. Plain hash chains and Merkle logs are 3–60 $\times$ faster but provide strictly weaker guarantees for clinical audit: hash chains give integrity without authenticity, Merkle logs localize tampering only to a batch root (not the specific action) and `rs_merkle` offers no rollback defense, and the HMAC chain cannot give a third-party auditor non-repudiation. Our per-entry-signed mode is the strong, slow extreme (every action independently and publicly attributable); our *batched* mode keeps per-entry localization and the hash chain while signing Merkle checkpoints, recovering throughput to within $\sim 3\times$ of the Merkle libraries (~ 0.5 M entries/s verify, in the same range as the 458,000/s end-to-end rate of §5.4). For a clinical record that must answer *which* action was altered and *who* signed it, that property bundle—not raw throughput—is the requirement.

5.6 Faster verification without weakening guarantees (E7)

Per-entry signing is the verifier’s *only* bottleneck—blake3 chaining alone runs at > 2 M entries/s (Table 2). Crucially, ed25519 verification is per-entry-independent, so it parallelizes and batches with *zero* change to the construction: same signatures, same keys, and the hash chain still localizes the tampered entry. The verifier therefore pre-computes signature validity off the sequential critical path—chunked `verify_batch` across `rayon` workers, with a per-signature fallback inside any chunk a batch rejects, so a single bad signature is still localized to its entry. On a 12-core machine:

This makes the strong, fully-attributable per-entry-signed mode a practical default rather than a slow extreme: it now verifies an order of magnitude faster while still naming the specific tampered action and the key that signed it.

verification method	M sig/s	speedup
sequential (per-entry)	0.04	1.0×
rayon parallel	0.29	8.3×
ed25519 batch	0.11	3.2×
parallel + batch (default)	0.82	23×

Table 3: Signature-verification speedup ($n=100k$, 12 cores, best of 3). The verifier now defaults to parallel+batch, taking per-entry-signed verification from the slowest construction to faster than our own batched mode and within $\sim 2\times$ of the Merkle libraries (Table 2)—while keeping per-entry public non-repudiation. Detection and localization are unchanged (E1: 26/26 detected, 20/20 localized); the result is cryptographically identical to sequential verification.

6 Security Argument

Integrity and non-repudiation reduce to two standard assumptions. *Chain integrity*: any modification of a recorded entry changes its prefix bytes, so detection fails only under a blake3 collision; tamper-evidence thus reduces to the collision-resistance of blake3 [10]. *Non-repudiation*: each entry’s hash is signed with ed25519, so a verifier holding the public key cannot forge a valid signature, and a recorded action cannot be repudiated unless the EUF-CMA security of ed25519 [1] is broken — a property a shared MAC cannot provide, since any MAC holder could forge. Per adversary tier: tiers (i) the agent and (ii) a co-located process cannot produce a self-consistent chain without the signing key, so TA1–TA5 are caught immediately; tier (iii) a privileged operator who also obtains the key can re-emit a consistent log, but cannot reproduce the external anchors (TA6 caught within k) and cannot re-sign under the real key seen by a verifier with a pinned public key (downgrade caught). Timestamps are recorded but never trusted for ordering — the sequence number is the order — so backdating (TA4) is a sequence/forgery violation, not a clock-trust question.

7 Discussion and Limitations

Relationship to HAARF. This work is the primary owner of HAARF category **C2** (end-to-end traceability and non-repudiable audit), and supports C8’s authorization-evidence requirements by making every gateway decision a signed, ordered, externally-anchored record [13].

Honest performance framing. Two pre-registered targets are not met on the development machine and we report them as such rather than re-scope them. Per-entry **fsync** makes chained writes ~ 4.71 ms (the correct fail-closed semantic; sub-millisecond is NVMe-class, §5.2), and a 10^6 -entry verify takes 2.18 s on one core (linear, parallelizable, §5.4). Both gaps are constant-factor and hardware/parallelism-bound, not algorithmic.

Other limitations. (a) Full-file rewrite with the signing key is defended only by anchoring and a pinned verifier key, not by the chain alone. (b) Signed external timestamps are future work; we record but do not trust wall-clock time. (c) Key management (HSM/KMS, attestation release) follows the QUOKKAGUARD pattern and is discussed, not benchmarked here. (d) A log with a trailing incomplete batch has uncheckpointed tail entries; the verifier surfaces this via a **signatures_checked** flag rather than silently treating the tail as verified. (e) Action streams are synthetic; realism of the modeled clinical workflow is a threat to external validity, addressed by the schema review in the proposal.

8 Conclusion

The record of what a clinical AI agent did is the foundation of every governance claim made about it, and until now this program left that record mutable. We made it tamper-evident: a unified, blake3-chained, ed25519-signed, externally-anchored, fail-closed log with a localizing verifier, detecting 100% of tampering at a cryptographic cost of 14.0 μ s/entry. Non-repudiable provenance is not the expensive part of trustworthy clinical AI — durability is — and durability is a deployment choice, not a research obstacle. The next layer in the program builds context-aware authorization (quwarden) on top of this evidence base.

Declaration of generative AI use

During the preparation of this work the author used *Claude (Anthropic)* as a coding and drafting assistant — for implementing and reviewing the Rust audit module under test-driven development, generating the figure-plotting and experiment-driver code, and drafting and copy-editing this manuscript. After using this tool, the author reviewed and edited all content and takes full responsibility for the publication. No generative-AI system was used to generate or fabricate experimental data, interpret results, or produce clinical conclusions, and no AI tool is listed as an author, as such tools do not satisfy authorship criteria.

Data and code availability

All code, the TamperBench generator and corpus, and result artifacts are available at github.com/quome-cloud/quledger under the Apache-2.0 license. All patient data are synthetic; no real protected health information was used.

Author contributions

J.S. conceived the study, designed and implemented the system, ran the experiments, and wrote the manuscript.

Competing interests

The author is affiliated with Quome Inc. The author declares no other competing interests.

Funding

This work received no specific external funding.

A Detailed result tables

Write mode	Median/entry	vs. plain	What it includes
plain v1 (no fsync)	42.6 μ s	1.0 $\times$	JSONL append, no durability
compute only (no fsync)	14.0 μ s	0.3 $\times$	blake3 + ed25519, no I/O
chained (+fsync)	4.95 ms	$\sim 116\times$	+ per-entry fsync
chained signed	4.71 ms	$\sim 110\times$	+ ed25519 per entry + fsync
chained signed batched	4.83 ms	$\sim 113\times$	+ batched checkpoint sigs + fsync

Table 4: E2 write cost per mode (medians, dev HW). The two-order-of-magnitude jump is per-entry `fsync`, not cryptography.

k	anchors	detection window	storage overhead	detected
10	2000	9	5.84%	yes
100	200	20	0.58%	yes
1000	20	13	0.058%	yes
10000	2	4013	0.006%	yes

Table 5: E3 anchor cadence sweep over a 20,000-entry log. Window $\leq k$ at every cadence; storage under 1% from $k=100$.

B Reproducibility

A single environment (Rust 2021 + Python 3 stdlib + `matplotlib`; no API keys) reproduces every result.

```

cargo build --release
python3 scripts/003-tamper-audit/e1_detection.py           # E1/E5 -> summary.json
cargo bench --bench audit_write                           # E2
cargo bench --bench audit_verify                          # E4
python3 scripts/003-tamper-audit/e3_anchor_sweep.py \
  > results/003-tamper-audit/e3_anchor/e3_anchor_sweep.csv # E3
python3 scripts/003-tamper-audit/make_figs.py             # figures
cd papers/003-tamper-audit && make paper                  # this PDF

```

All randomized steps use seed 42; result artifacts are written under `results/003-tamper-audit/`.

References

- [1] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.
- [2] European Parliament and Council. Regulation (eu) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act). Official Journal of the European Union, 2024. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>.
- [3] Google. Trillian: A transparent, highly scalable and cryptographically verifiable data store. <https://github.com/google/trillian>, 2017.
- [4] Google Cloud. Confidential space. Google Cloud documentation, 2024.
- [5] Stuart Haber and W. Scott Stornetta. How to time-stamp a digital document. *Journal of Cryptology*, 3(2):99–111, 1991.
- [6] John Kelsey, Jon Callas, and Alexander Clemm. Signed syslog messages. RFC 5848, IETF, 2010.
- [7] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate Transparency. RFC 6962, IETF, 2013. <https://www.rfc-editor.org/rfc/rfc6962>.
- [8] Ralph C. Merkle. A digital signature based on a conventional encryption function. In *Advances in Cryptology — CRYPTO ’87*, pages 369–378. Springer, 1988.
- [9] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). NIST AI 100-1, 2023. <https://doi.org/10.6028/NIST.AI.100-1>.
- [10] Jack O’Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O’Hearn. BLAKE3: One function, fast everywhere. <https://github.com/BLAKE3-team/BLAKE3-specs>, 2020.
- [11] James Schwoebel. Pristine weights, poisoned goals: Intent-mutation attacks on self-improving clinical agents and attestation-rooted harness defenses, 2026. Quome, QUOKKAGUARD program. <https://github.com/quome-cloud/quanchor>.
- [12] James Schwoebel, Ingrida Semenec, Jenia Rousseva, Martin G. Frasch, Rome Thorstenson, and Manish Bhatt. Beyond injection detection: A positive-security prompt firewall that closes the scope and PHI gap SOTA classifiers miss in healthcare. *medRxiv*, 2026. Preprint, <https://doi.org/10.64898/2026.06.04.26354950>.
- [13] Jim Schwoebel, Martin Frasch, Art Spalding, Ed Sewell, Phil Englert, Ben Halpert, Collin Overbay, Ingrida Semenec, and Joel Shor. HAARF: Healthcare AI agents regulatory framework — a comprehensive security verification standard for autonomous AI systems in clinical environments. medRxiv preprint, <https://doi.org/10.64898/2026.04.09.26350519>, April 2026.
- [14] U.S. Department of Health and Human Services. HIPAA security rule: Technical safeguards. 45 CFR § 164.312, 2003. <https://www.ecfr.gov/current/title-45/subtitle-A/subchapter-C/part-164/subpart-C/section-164.312>.

- [15] U.S. Food and Drug Administration. Electronic records; electronic signatures. 21 CFR Part 11, 1997. <https://www.ecfr.gov/current/title-21/chapter-I/subchapter-A/part-11>.
- [16] U.S. Food and Drug Administration. Marketing submission recommendations for a predetermined change control plan for AI-enabled device software functions, 2024.
- [17] Ziqi Zhang, Chen Gong, Yifeng Cai, Yuanyuan Yuan, Bingyan Liu, Ding Li, Yao Guo, and Xiangqun Chen. No privacy left outside: On the (in-)security of TEE-shielded DNN partition for on-device ML. *arXiv preprint arXiv:2310.07152*, 2023.